

TimeSlips Mobile App

Detailed Design Document

Team 6121: Joel Sari, Joshua Dietzenbach, Viktoriya Petrova,
Subhan Aamir, Esteban Sanint

Client: Anne Basting & Dhruv Gami

Repository: <https://github.com/Joel-sari/TimeSlipsMobileAI.git>

Table of Contents

<u>Table of Figures</u>	4
<u>Terminology</u>	5
<u>Introduction</u>	7
Background	7
Document Summary	7
<u>Static System Architecture</u>	8
Introduction	8
Rationale	8
Static Architecture	9
<u>Dynamic system Architecture</u>	11
Introduction	11
Explanation	12
<u>Data Design</u>	13
Introduction	13
Database Use	13
File Use	15
Data Exchange	15
Security and Privacy Implications	15
<u>UI Design and Appendices</u>	17
Introduction	17

Home Screen	17
Prep Mode	19
Engage Mode	21
History Mode	24
Profile Page	26
Appendix	28

Table of Figures

Figure Number	Figure Title or Caption	Page
<u>Figure 1</u>	Static System Architecture: Component Diagram	8
<u>Figure 1.1</u>	DynamoDB Data Model: Document Structure Diagram	10
<u>Figure 2.1 - 2.2</u>	Home Screen App User Flow	16
<u>Figure 3.1 - 3.5</u>	Prep Mode App User Flow	18
<u>Figure 4.1 - 4.6</u>	Engage Mode App User Flow	20
<u>Figure 5.1 - 5.3</u>	History Mode App User Flow	23
<u>Figure 6.1 - 6.3</u>	Profile Page App User Flow	25

Terminology

Term	Definition / Context
Amplify	AWS Amplify is used to provision and manage the cloud backend resources for the application, including the API stack, storage resources, and supporting infrastructure.
API Gateway	Amazon API Gateway exposes the REST endpoints used by the mobile client and routes protected requests into backend Lambda handlers.
Appwrite	Appwrite handles user authentication and session management for the mobile application. The frontend obtains a JWT that is validated before protected API calls are accepted.
DynamoDB	Amazon DynamoDB is the primary NoSQL database used to store users, prompts, sessions, recordings, reflections, and related workflow records.
Expo	Expo provides the managed React Native workflow used to build and run the TimeSlips mobile application across devices.
Global Secondary Index (GSI)	A DynamoDB secondary index that supports alternate query patterns without changing the table's primary key. Examples in this system include email-based, facilitator-based, session-based, and status-based queries.
JWT	JSON Web Token used by the frontend to prove the identity of the authenticated user when calling protected backend endpoints.
Lambda	AWS Lambda functions contain the backend business logic for users, prompts, storytellers, sessions, recordings, reflections, and deletion-request handling.
Lambda Authorizer	A custom authorization layer attached to API Gateway that validates the caller's token and blocks unauthorized requests before they reach protected Lambda handlers.

NoSQL	A non-relational data model that stores records in flexible key-value or document-oriented structures rather than normalized relational tables.
PII	Personally identifiable information. In this system, facilitator details, storyteller names, audio recordings, and transcripts are treated as sensitive data.
RBAC	Role-Based Access Control. The application distinguishes at minimum between facilitator and admin capabilities.
REST	Representational State Transfer. The backend is exposed as RESTful endpoints used by the mobile app.
S3	Amazon S3 stores binary assets such as uploaded recordings and other media needed by the app.
Transcription Pipeline	The application's audio-to-text workflow that processes recorded storytelling audio and stores the resulting transcript for later review.
Whisper AI	Whisper AI is the transcription model used to convert recorded storytelling audio into text. In TimeSlips, it supports the Engage/History workflow by generating transcripts that facilitators can review and confirm after a session.

1. Introduction

1.1 Background

TimeSlips is a nonprofit organization that uses guided creative storytelling to improve connection, expression, and engagement for older adults, including many living with dementia. In practice, facilitators need a lightweight way to prepare prompts, manage storytellers, start sessions, capture recordings, review transcripts, record reflections, and later revisit completed sessions. The TimeSlips Mobile App addresses that need with a cross-platform mobile workflow centered on Prep, Engage, History, and admin oversight.

Compared with the original Version 1 design, the current architecture now reflects the implemented authentication and backend flow used by the team. The updated design therefore focuses on the mobile client, Appwrite-based authentication, API Gateway with a Lambda authorizer, Lambda-based business logic, DynamoDB persistence, S3 media storage, and the session transcription pipeline.

1.2 Document Summary

This Detailed Design Document describes the technical architecture, data model, and design decisions for the TimeSlips Mobile App. It is structured as follows:

- Terminology: updates outdated platform and security terminology so it matches the current implementation.
- Introduction and Summary: updates the scope of the system and removes references that described later sections as future work.
- Static System Architecture: documents the current service layout, the main technical rationale, and the security model.
- Data Design: documents the DynamoDB schema, key/index design, file storage usage, and data exchange patterns used by the application.

2. Static System Architecture

2.1 Introduction

The TimeSlips Mobile App uses a serverless, cloud-based architecture designed to keep the mobile client lightweight while centralizing security, persistence, and workflow logic in managed AWS services. The frontend is a React Native application built with Expo. It communicates with a REST backend exposed through API Gateway. Protected requests pass through a custom Lambda authorizer and are then routed to Lambda handlers that read and write application data in DynamoDB and store media in Amazon S3.

This architecture was selected to support cross-platform mobile development, role-aware access control, scalable data storage, and asynchronous processing for recording and transcription workflows. The same architecture also supports future maintainability because core services are loosely coupled: the client, auth layer, API layer, compute layer, and storage layer can each evolve without rewriting the entire system.

2.2 Rationale

This section details the rationale behind the team's architectural choices and the security considerations taken into account.

- **Frontend - React Native with Expo:** React Native allows a single mobile codebase for the core facilitator workflows, while Expo simplifies device access, local development, and app iteration.
- **Authentication - Appwrite:** Appwrite provides the active authentication flow used by the application and issues session credentials that the frontend uses when calling protected backend resources.
- **API Layer - API Gateway + Lambda Authorizer:** API Gateway exposes the backend as REST endpoints, and the authorizer provides a consistent gate for validating authenticated requests before business logic runs.
- **Backend Compute - AWS Lambda:** Lambda handlers support the project's serverless approach and isolate business logic for users, prompts, storytellers, sessions, recordings, reflections, and deletion requests.
- **Database - Amazon DynamoDB:** DynamoDB supports the app's flexible and session-centric data model. Single-table joins are not required, and alternate access patterns are handled using targeted GSIs.
- **Object Storage - Amazon S3:** S3 stores audio recordings and other binary assets outside the database while still integrating cleanly with backend workflows.
- **Transcription Workflow - Whisper AI:** Audio recordings are processed using Whisper AI so facilitators can review the story text after recording rather than manually retype it.

The transcript is intended to preserve the storyteller's speech as closely as possible, including mistakes or informal phrasing, so facilitators can review the story accurately.

Security Considerations:

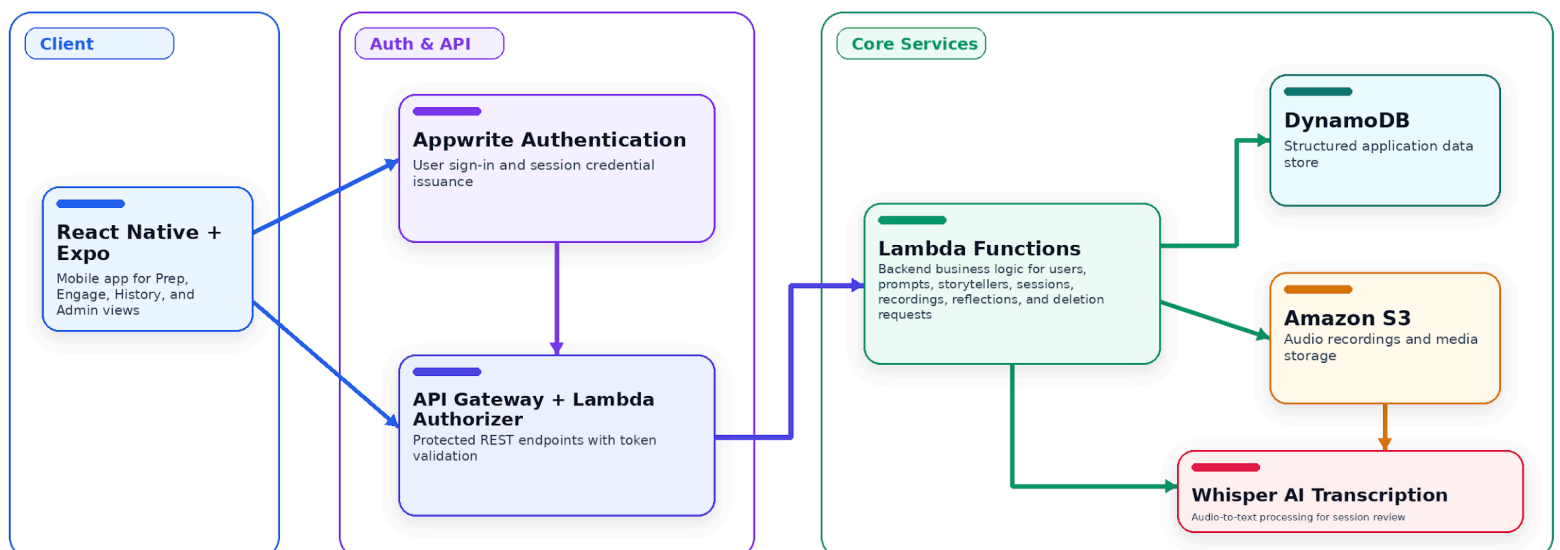
- Authentication and authorization: protected requests require a valid authenticated session token, and the API layer enforces role-aware access before sensitive actions are allowed.
- PII protection: storyteller names, facilitator information, recordings, and transcripts are treated as sensitive data and are only exposed to authorized users within the system.
- Data at rest: structured data is stored in DynamoDB and media in S3, both under AWS-managed storage controls.
- Data in transit: client-to-backend communication uses HTTPS with JSON request/response bodies for structured data.
- Least-privilege backend design: individual Lambda handlers are responsible for only the subset of operations associated with their endpoint domain.

2.3 Static Architecture

Figure 1 shows the updated static component view. The mobile client authenticates through Appwrite, sends protected requests into API Gateway, and relies on Lambda-based backend handlers for business logic. The backend persists structured records in DynamoDB, stores media in S3, and invokes the transcription workflow for recorded sessions.

Figure 1. Static System Architecture

TimeSlips Mobile App



<i>Layer</i>	<i>Main Components</i>	<i>Purpose</i>
Client	React Native + Expo	Provides the user interface for prep, engage, history, and admin workflows.
Authentication	Appwrite	Authenticates users and provides session credentials for protected access.
API + Authorization	API Gateway + Lambda Authorizer	Exposes REST endpoints and validates protected requests before backend execution.
Business Logic	Lambda functions	Implements the core workflow logic for users, prompts, storytellers, sessions, recordings, reflections, and deletion requests.
Structured Storage	DynamoDB	Stores the persistent operational data used by the app.
Media Storage	Amazon S3	Stores audio recordings and related binary media.
Processing	Transcription pipeline/service	Converts recorded audio into reviewable transcript text.

3.3 Explanation

When the facilitator opens the Home screen, the client triggers prompt loading through the shared API client. Before the request is sent to the backend, the client retrieves (or reuses) a short-lived Appwrite JWT via `getJWT()`. The API client then calls the REST API Gateway with the JWT in the **Authorization: Bearer <jwt>** header. API Gateway invokes the **JWT Authorizer Lambda**, which validates the token against Appwrite and, on success, returns an Allow policy with a **principalId** representing the signed-in user. The request is then routed to the **Prompts Lambda**, which reads the prompt data from the **Prompts DynamoDB table**. If the prompt references an S3 image key, the Prompts Lambda generates a **pre-signed S3 URL** and includes it in the response so the mobile app can render the image without direct S3 credentials.

In parallel, the client refreshes favorites using the same authenticated request path **GET /favorites**. When the user taps the Prompt card, the app opens the Prompt Detail Modal using already-fetched prompt data (no network call is required). When the user taps the heart icon, the client updates favorites optimistically in local state, then issues a **POST /favorites** request. The Favorites Lambda writes a new item to the **FavoritePrompts DynamoDB table** using the authenticated user id (**principalId**) + prompt id as the key. If the write succeeds, the UI stays favorited; if it fails, the optimistic update is rolled back to preserve consistency.

Data Flow: Prompt metadata is stored in DynamoDB (promptId, title, body, category, tags, imageKey). Image binaries live in S3, but the client never accesses S3 directly; instead, the backend returns a time-limited pre-signed URL per request. Favorite state is stored as a user-scoped DynamoDB record (userId + promptId), ensuring favorites are tied to the authenticated facilitator and preventing the client from spoofing another user's id.

Key reliability/Security Notes: Every backend request is authenticated through the authorizer, and the server derives the user identity from **principalId** rather than trusting user input. JWT caching avoids races and reduces repeated auth traffic during the Home screen's parallel loading.

3. Data Design

3.1 Introduction

The TimeSlips data model is centered on facilitation workflows rather than on heavy relational joins. DynamoDB is therefore a good fit because it supports a flexible record structure, simple primary-key access for the main entities, and targeted GSIs for alternate access patterns such as facilitator-based lookups, email lookup, category filtering, session-based retrieval, and status-based request review.

3.2 Database Use

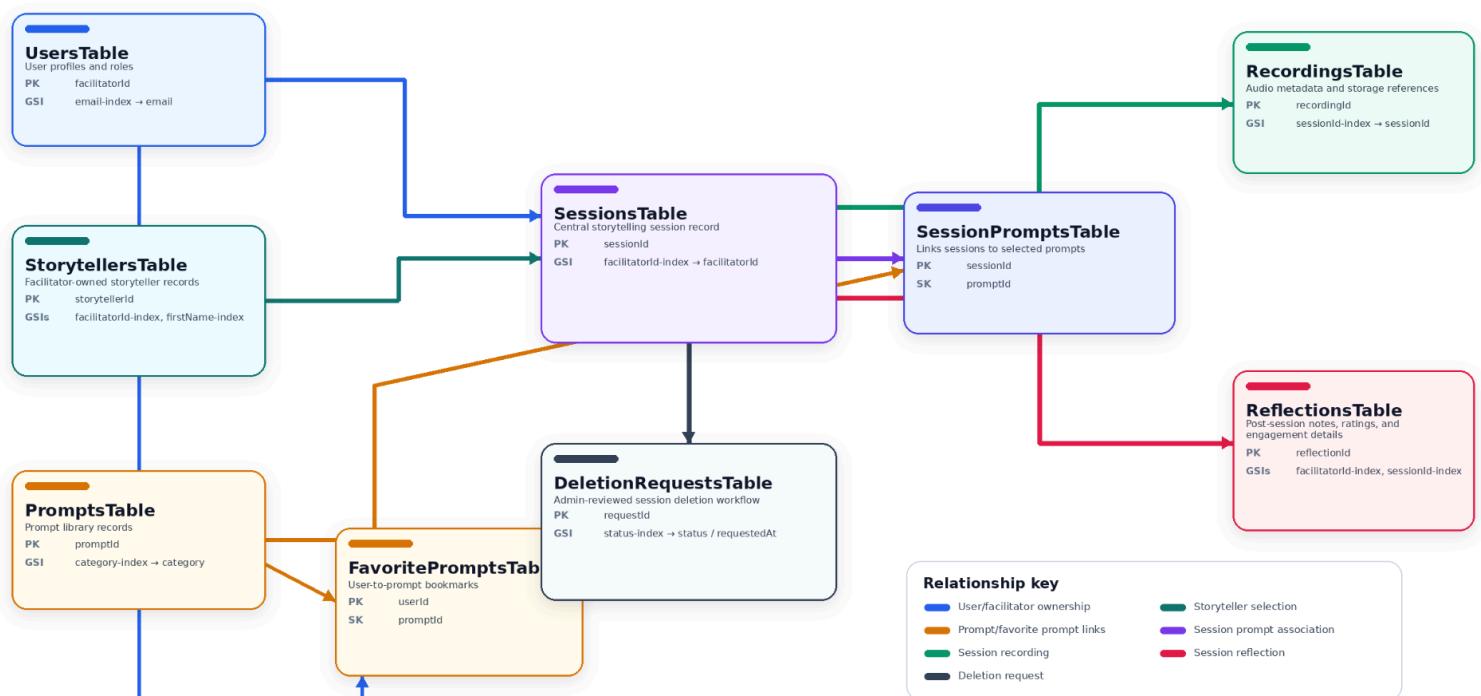
The current backend uses the following core DynamoDB tables. The table below summarizes the primary keys and the most important global secondary indexes visible in the current stack configuration.

Table	Partition Key	Sort Key	Important GSIs
UsersTable	facilitatorId	—	email-index → PK: email
StorytellersTable	storytellerId	—	facilitatorId-index → PK: facilitatorId; firstName-index → PK: firstName
SessionsTable	sessionId	—	facilitatorId-index → PK: facilitatorId
PromptsTable	promptId	—	category-index → PK: category
FavoritePromptsTable	userId	promptId	None
RecordingsTable	recordingId	—	sessionId-index → PK: sessionId
ReflectionsTable	reflectionId	—	facilitatorId-index → PK: facilitatorId, SK: createdAt; sessionId-index → PK: sessionId
SessionPromptsTable	sessionId	promptId	None
DeletionRequestsTable	requestId	—	status-index → PK: status, SK: requestedAt

At the entity level, the model is session-centric. Before a session begins, facilitators interact with storytellers, browse and select prompts, and manage favorite prompts, while administrators manage users, role-based access, and prompt governance. During and after a session, the system stores the session record itself along with related recordings, reflections, prompt associations, and deletion requests. Figure 1.1 illustrates the core relationships among these records.

Figure 2. Core Data Model Relationships

Session-centric DynamoDB entity view



Sessions are the center of the model. Facilitators work with storytellers and prompts before a session; the system then stores related recordings, reflections, prompt associations, and deletion-request records.

3.3 File Use

Almost all applications use some standard files, and the TimeSlips system leverages Amazon S3 to store binary objects utilized by the frontend and backend.

File Type	Likely Format	Storage Location	Purpose
Audio recordings	.m4a / .wav	Amazon S3	Stores recorded storytelling audio captured during Engage Mode.
Prompt images	.jpg / .png	Amazon S3	Stores visual prompt media associated with the prompt library or curated content.
Whisper transcript outputs	text / JSON-backed processing output	DynamoDB-backed workflow records	Stores the text result of the transcription workflow for facilitator review and later session history access.

3.4 Data Exchange

The primary data exchange pattern is mobile-to-backend communication over HTTPS using JSON request and response bodies. The mobile client sends authenticated requests to the REST API, and protected endpoints are validated before they are routed to the correct Lambda handler. Structured records are then stored in DynamoDB, while larger binary media such as recordings are handled through S3-backed flows.

Operationally, the exchange model supports three common categories of data movement:

- Read and write operations for users, prompts, storytellers, sessions, reflections, and deletion requests.
- Media upload and retrieval flows for recordings and prompt-related assets.
- Asynchronous transcript processing in which recorded audio is converted into reviewable text and then surfaced back through the session/history workflow.

3.5 Security and Privacy Implications

- Only authenticated users can access protected application data.

- RBAC limits access to admin-only flows such as role management, prompt governance, and deletion-request processing.
- PII and sensitive storytelling artifacts remain internal to the system and are not designed for public sharing.
- The storage design separates structured metadata from large media assets, which reduces unnecessary database bloat and supports cleaner lifecycle handling.

4. UI Design and Heuristic Analysis

4.1 Introduction

In this section, we present the user interface, design heuristics, and design considerations of the TimeSlips mobile app. There are five categories of screens: home, prep mode, engage mode, history, and profile. Each category will have a description of the screens, an analysis of the heuristics, and other design considerations.

When opening the application, the user is taken to the home screen. If the user is not already signed in, they are taken to a sign-in page, where they can enter their email.

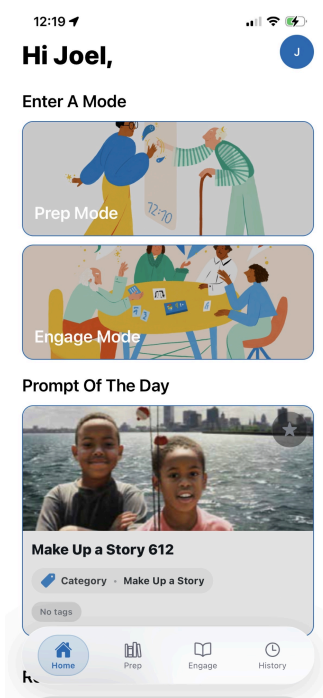


Figure 2.1 Home Screen First Half

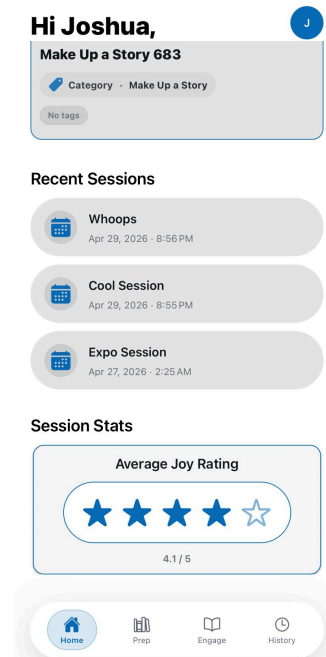


Figure 2.2 Home Screen Second Half

4.2 Home Screen

The home screen (Figures 2.1 and 2.2) is laid out as an introductory step for the user. In the top right corner is the profile icon, which can be clicked into. It has the two main modes listed as cards, with images to support. Then, there is a randomized prompt of the day, with a prompt card displayed. Then, there is a list of the three most recent sessions, with the ability to take the user to those sessions in history. There are also session stats of the average joy rating and

social connection scale of the user's sessions. Lastly, the tab bar is available at the bottom of all of the main menus of the four sections: home, prep, engage, and history.

The heuristics applied on the home screen are recognition rather than recall, flexibility and efficiency of use, and aesthetic and minimalist design. Recognition rather than recall is prioritized through the labeling of each area of the home screen: enter a mode, prompt of the day, recent sessions, and session stats. Flexibility and ease of use are prioritized by having a tab bar up at all times, in case experienced users want to quickly navigate to a screen. Furthermore, the recent sessions help the user quickly navigate to sessions that occurred recently without the need to search for them. Aesthetic and minimalist design is prioritized by not having too many things in each section. We focused on limiting recent sessions, having one prompt of the day, and having the session stats horizontally scroll to avoid overcrowding and overstimulation.

When designing the home screen, the goal was to create an introduction to the app itself, with the main modes listed, ways to view an already curated prompt, quickly see sessions, and view some quick stats. We focused on following the brand identity of TimeSlips, with their colors and imagery. There is a focus on vertical alignment, with each section title bolded and larger to give hierarchy, alignment, and proximity of similar items. The tab bar focuses on a drop shadow to distinguish it being in the foreground, instead of an element of the page.

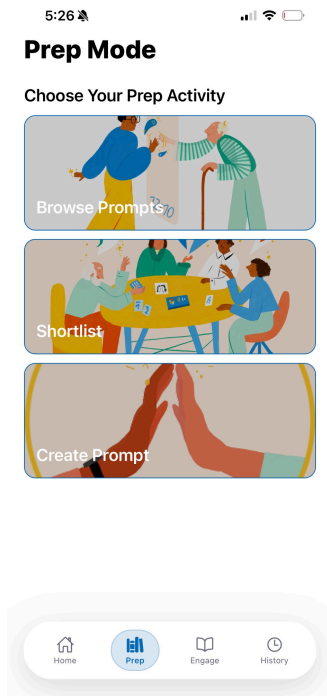


Figure 3.1 Prep

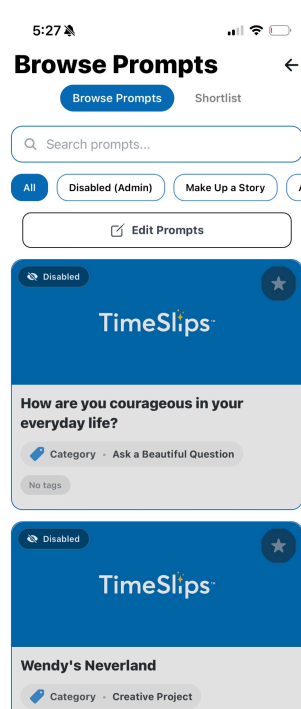


Figure 3.2 Browse Prompts

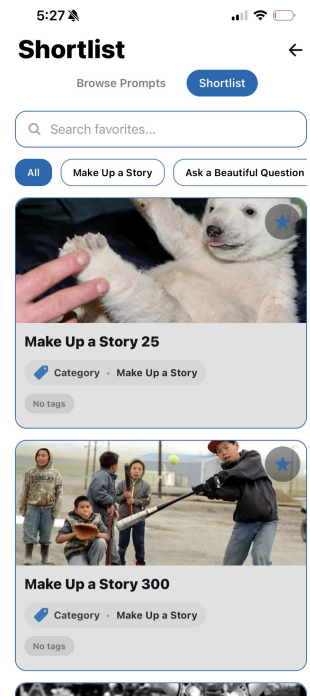


Figure 3.3 Shortlist

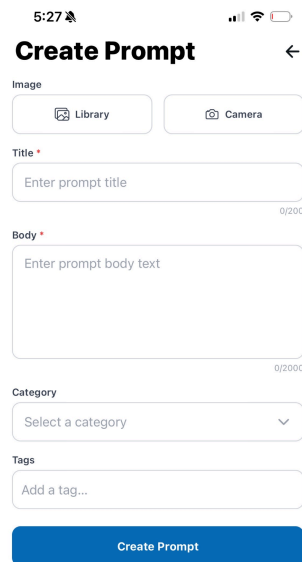


Figure 3.4 Create Prompt

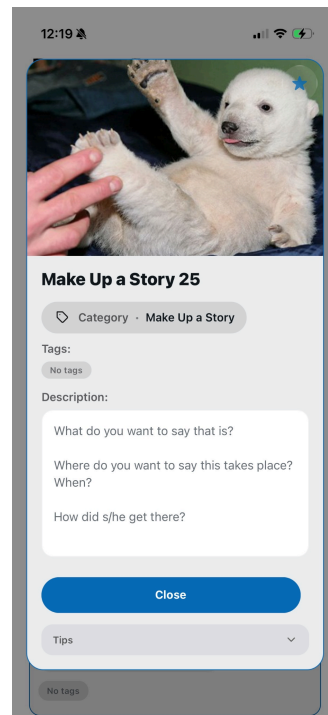


Figure 3.4 Prompt Details

4.3 Prep Mode Screens

The prep mode screen (Figure 3.1) opens to two to three options (depending on whether the user is an admin): browse prompts, shortlist, and create a prompt. The browsing prompts screen (Figure 3.2) contains all the prompt cards, with the ability to be clicked into to learn more description (Figure 3.5), as well as the option to favorite the prompt cards and add them to the user's shortlist. There is also a search bar to search by prompt name, as well as the ability to sort by categories and tags. On the shortlist screen (Figure 3.3), the user can see favorited prompts, unfavorite prompts, and have the same search features as browse prompts. These two screens can be switched back and forth with a tab bar at the top. There is also a back arrow to go back to the Prep options. Lastly, the create a prompt screen (Figure 3.4) has different text and dropdown fields that the user can fill in, including title, category, tags, description, and image. Some of these have mandatory icons to let the user know that they have to be filled in to continue.

The heuristics applied to the prep mode screens are match between system and real world, user control and freedom, and flexibility and efficiency of use. Match between the system and the real world is prioritized through familiar language and symbols, including browsing, prompts, categories, and tags. Furthermore, the ability to shortlist is done through clicking a star, which is a universal symbol of favoriting or prioritizing. User control and freedom are given through the easy ability to unfavorite prompts by clicking the star again. There is also a back arrow in the browse prompts and shortlist screens, as well as a close button when viewing prompts. Flexibility and efficiency of use are helped by the ability to choose labels and add custom tags to prompts when creating. This helps the ease of use of searching for specific types of prompts in either the browse prompts and shortlist menu, as they are fields that can be selected to sort by. The ability to search by prompt name also helps if a user is looking for a specific prompt.

When designing the prep screens, the goal was for easy searchability and way to save prompts. We focused on having search features, categories, and labels for every prompt in the library to narrow down the search. Similarly, the prompts can be gone into and viewed in more detail. We prioritized visual accessibility so the favorite star can be seen on any image by adding a circle around it. The prompts are also aligned and can be scrolled through. We valued consistency through the prompts, so making sure the prompt creation page had mandatory fields helped to make sure that all prompts while searching are to the same standard. Visually, the prompts stand out from the background and are easily readable with contrast.

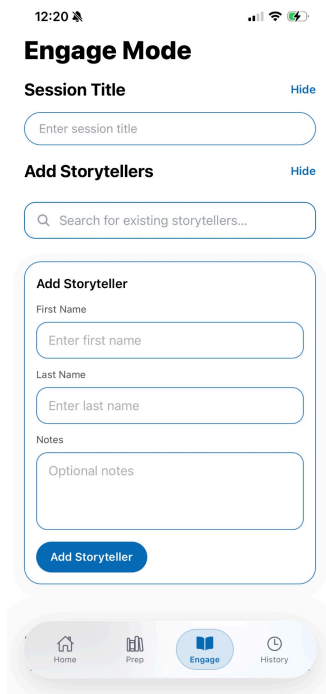


Figure 4.1 Session Creation

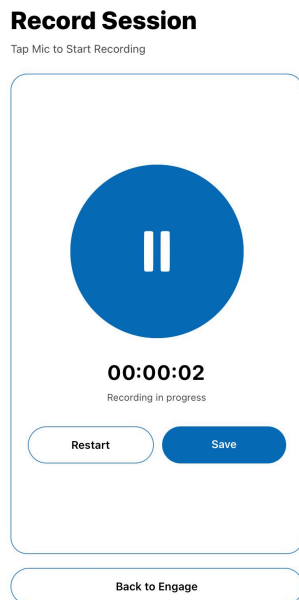


Figure 4.2 Record Session

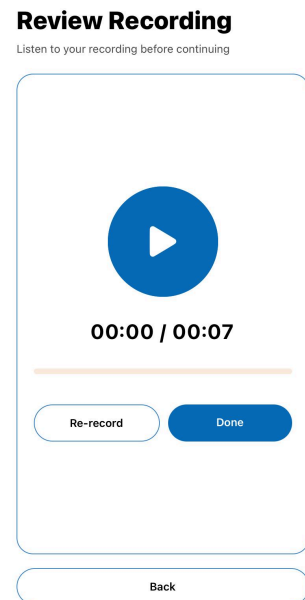


Figure 4.3 Review Recording

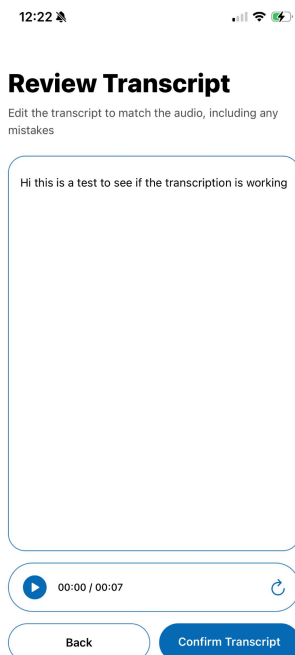


Figure 4.4 Review Transcript

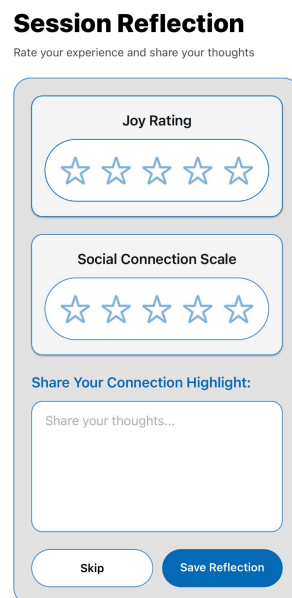


Figure 4.5 Session Reflection

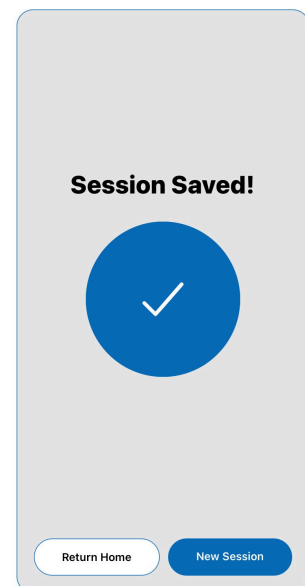


Figure 4.6 Saved Session

4.4 Engage Mode Screens

Engage mode opens to the session creation screen (Figure 4.1). In the session creation screen, the user has a text field to fill in the Session Title; the ability to add storytellers through searching through existing storytellers, in which they have the option to add, edit, or delete them, or add a storyteller by first, last name, and description; and select a prompt from either 5 recommended options or from the user's favorites. There is an option to start a session, which can only be done once at least one storyteller is added, and a prompt is selected. There is also an option to clear out all of the fields. Each section has the ability to be hidden. In the record session screen (Figure 4.2), there is a big circle that is either a microphone, pause symbol, or play symbol. There is a timer beneath that telling the user how long the recording is. There is a button to save the recording and a button to restart the recording. There is also the option to go back to the Engage screen. In the review recording screen (Figure 4.3), there is a play/pause button to listen back to the recording. There is a timeline and a timer to show how far in the recording the user is, as well as two buttons to either re-record or be done listening. There is also a back button. In the review transcript screen (Figure 4.4), there is a ready to transcribe screen with a button as well as a text pop-up to start transcription. There is a loading state while the transcription is occurring, and the text pops up in the text box after. There is also the ability to listen back to the audio beneath the text box, as well as a back and confirm transcript button. In the session reflection screen (Figure 4.5), there are three different fields. The first two are joy and session connection scale, which can be filled in by clicking the number of stars beneath each, which all fill in automatically to the left. There is also a text box to write the connection highlight. There is a button to skip this or save the reflection. There is then the session saved screen (Figure 4.6), which confirms that the session is completed and saved with text and a check mark, and has two buttons to start a new session or return to the home screen.

The heuristics applied to the engage mode screens are visibility of system status, user control and freedom, and error prevention. For visibility of system status, there are constant updates on progress throughout the screens. In the session creation screen, it lets the user know if they have to fill in more fields, as well as hides prompts once one is selected, and puts a list of storytellers added. The record screen changes the icon of the record button when recording, including filling in the button, to let the user know the status of recording, as well as the time. There is a timer bar when listening back for the user to know where they are in the recording. There is a loading screen and progress bar when the transcript is loading. Lastly, the session saved screen helps let the user know that their session is saved, so there is no doubt about the progress. For user control and freedom, there are many ways to cancel or undo tasks. In the session creation screen, the user can remove and even delete users from the session, unadd the prompt, and even have a button to clear all fields. In record session, the user can restart recording or return to the engage screen. In review recording, the user can go back to continue recording or re-record the entire thing. In review transcript, the user can go back to the review recording screen. For error prevention, there are ways to prevent errors, as well as ways to issue warning messages. In the session creation screen, the user is not able to move on until a prompt and one storyteller is added. Furthermore, there is a default title with storyteller's name and date for sessions if the user forgets or doesn't want to add one in. There are pop-up

confirmation messages for when the user presses clear on the session creation screen, when the user presses restart on the record session, and when the user presses re-record on the review recording.

When designing the engage screens, the focus was on progressive disclosure and consistency. The session creation screen creates a hierarchy view, with each page collapsible to create a better view. The progressive disclosure comes with each screen step, with the completion of each step needed to move on to the next. This helps to stop users from getting overloaded with information and makes sure that steps are completed in the correct order. There is also a consistency with the box, which helps with accessibility and being able to read text. Also, having the redo buttons not filled in and the clear button being red helps to convey they are not main options and are undo options.

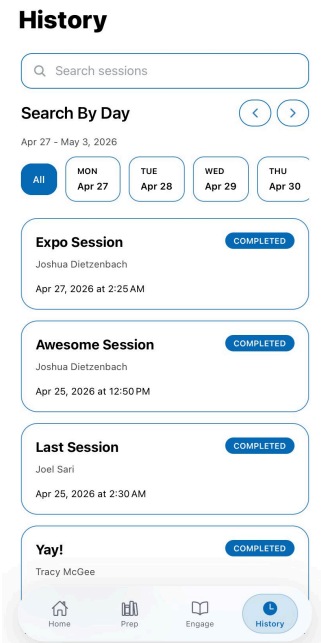


Figure 5.1 History

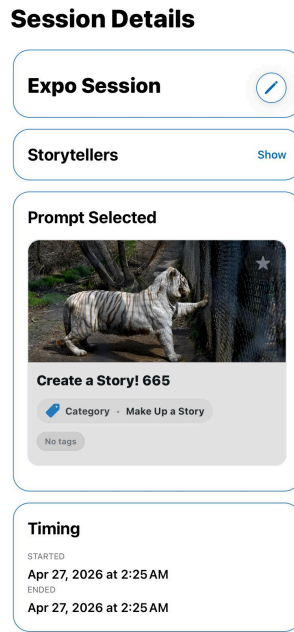


Figure 5.2 Session Details

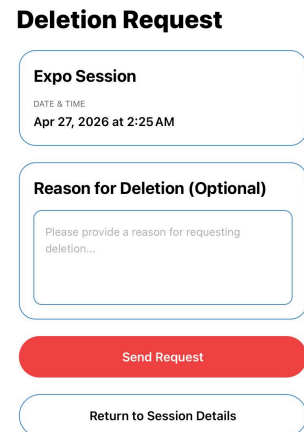


Figure 5.3 Deletion Request

4.5 History Screens

The history screen (Figure 5.1) opens to a screen with a list of all sessions, with each listing the title, storytellers, and time. There is a search sessions bar, as well as the ability to search by day through a weekly calendar sorting field. The user can click on a session card, and it will take the user to the session details screen (Figure 5.2). In session details, it lists the session title, storytellers, prompt selected, timing, reflection, audio playback, and transcript. The session title can be edited with an edit button. Storytellers, reflection, and transcript can be shown or hidden, and they resemble their engage mode counterparts. There is an edit button for reflection, and there are two buttons to return to history or request deletion. In the deletion request screen (Figure 5.3), there is the session title and time listed, as well as a text box to fill in the reason for deletion. There is a send request button that has a pop-up warning, and changes to “sending...” while loading. There is a pop-up confirmation that it was sent to an admin when it is sent. There is also a button to return to the session details screen.

The heuristics applied to the history screen are user control and freedom, consistency and standard, and flexibility and efficiency of use. For user control and freedom, there is a button to clear changes when editing the reflection and transcript. There is also a button to return to session details instead of forcing the user to make a deletion requisition. For consistency and standard, the same terminology is used in the engage mode, and the same cards are used. Similarly, all icons used in it are standard to the industry, with the play button, edit icon, and search bar. For flexibility and efficiency of use, the search bar enables the user to search by session name or storytellers, so it is easy to jump to a specific session. Also, the ability to sort by day through a calendar sort makes it easy for users to find a session from a specific date.

When designing the history page, the focus was on consistency and alignment, as well as organization. There is a consistency between the engage screens and what can be viewed in the session details, which helps with consistency. The sessions are aligned vertically and in their own boxes with quick information. There are also different sections in the session details to distinguish between parts, with bolded titles. Accessibility and contrast are prioritized through boxed sections and consideration of readability.

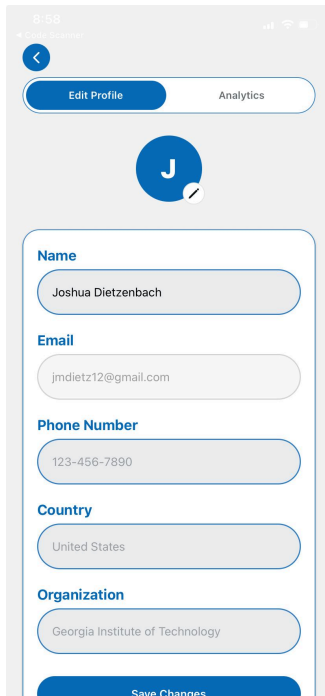


Figure 6.1 Profile

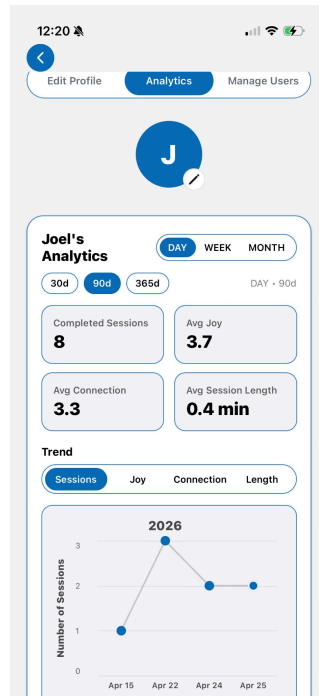


Figure 6.2 Analytics

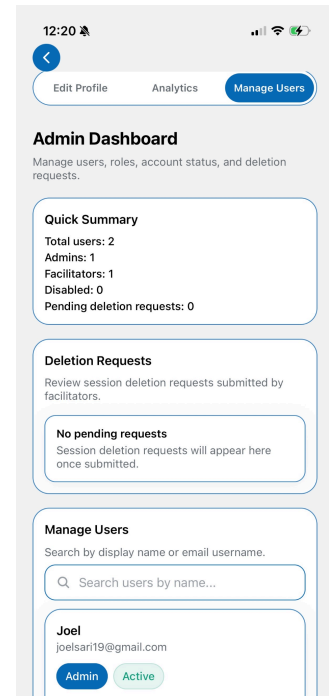


Figure 6.3 Manage Users

4.6 Profile Screens

The profile screen (Figure 6.1) has different editable fields, including name, email, phone number, country, and organization, as well as a button to save the changes. There is an edit icon next to the profile to change the theme colors of the app. The analytics screen (Figure 6.2) contains a box for completed sessions, average joy rating, average connection, and average session length. There is a graph that can show each of those values broken down by either day, week, or month in the span of the last 30, 90, or 365 days, depending on the sliders selected. In the manage users screen (Figure 6.3), which is only accessible by admins, there is a summary of users, a list of pending deletion requests, and a manage users section, where users have their name, email, and whether they are admin, facilitators, or active. This has a search bar to search for a specific user.

The heuristics applied to the profile screens are consistency and standards, recognition rather than recall, and aesthetic and minimalist design. For consistency and standards, the same terms can be found in the reflection and sessions. The graphing and ways to sort it out also follow standard graph conventions, with dates listed on the x-axis, and the scale listed in the y-axis. For recognition rather than recall, each field is properly titled, and the graph highlights what data the user is looking at. Each section of the manage users screen has a descriptive title and description of what occurs. For aesthetic and visual design, each field is concise on the profile screen, the different fields and filters all fit in the user's view on the analytics screen, and there is a quick summary on the manage users screen.

When designing the profile screens, the focus is on consistency and alignment. There is consistency with the analytics page and the way the other screens interact. The alignment is also vertical, and has important titles and information bolded. Furthermore, there is the ability to change the theme in the profile screen, which creates UI customization for the entire app. This can help users be able to have more contrast if needed, or if a user suffers from color blindness, they can choose a color they see better.

Appendix

Section 1: Team Member Profiles

Member 1

Name: Subhan Aamir

Role: Backend Developer

Technical Contributions:

- Role Based Access - Defined a tiered role hierarchy, separating standard facilitators from administrators, and implemented the backend logic to validate role permissions before executing restricted operations.
- Session CRUD - Developed the create, read, update, and delete (CRUD) operations for sessions in the backend, communicating with AWS DynamoDB.
- Exclusive Prompt and Library Governance - Allowed admins the exclusive authority to delete prompts from the global library so Beautiful Questions collection remains curated. Admin can also view all prompts, disable/remove from the facilitator's view.
- UI Clean up and Filters - Refactored the navigation tabs for the app for a better UX feel, also cleaned up unnecessary data displayed on session cards in the history page. Added better filters and tag filtering for better admin and facilitator experience in the engage browse prompts.

Contact Information: saamir7@gatech.edu

Member 2

Name: Esteban Sanint

Role: Full Stack Developer

Technical Contributions:

- Spearheaded the ideation and creation of the Detailed Design Document v1.
- created the AWS Amplify project and environment configuration as the team's foundation for cloud deployments (SETUP-02), unblocking everyone else's backend and infra work.
- Authored the DynamoDB data model documentation covering all schemas (SETUP-04) so the team had a single source of truth for table structure, keys, and relationships before any handler code went in.
- Built the Prompt Library list screen (PREP-02): a scrollable list/grid driven by the TimeSlips Beautiful Questions export, rendering prompt text + metadata with a pending-state fallback when prompts haven't been delivered.
- Owned the Custom Prompts feature end-to-end (PREP-06, PromptsTable Lambda): added POST /prompts, PUT /prompts/{id}, and DELETE /prompts/{id} to the prompts handler, plus the "Create Prompt" entry from Prep Mode, the form (title/body/category/image upload + validation), and the edit/delete UI. Distinguished custom vs. TimeSlips prompts via a source field so facilitators can only edit/delete their own, and made custom prompts favorable like any library prompt.
- Designed and shipped the team's CI/CD pipeline on GitHub Actions (INFRA-02): a workflow that runs lint, type-check, and build on every PR + push to main; an Amplify sandbox/staging deploy trigger on merge; full environment-variable + secrets

configuration in repo settings; and branch protection rules that block merges until CI passes.

- Led a Sprint 2 code-hygiene and scaffolding pass (INFRA-03) to clear technical debt before the Engage/History push: drove lint to zero (fixed the blocking role-gaurd.tsx apostrophe error and resolved all 7 warnings), renamed typo files (role-gaurd.tsx → role-guard.tsx, transcript-review.tsx → transcript-review.tsx), removed Expo starter leftovers (hello-wave.tsx, parallax-scroll-view.tsx, themed-text/view.tsx, app/modal.tsx), and stripped stray console.logs from auth and global provider layers.
- Aligned project documentation with the actual codebase so stakeholders weren't reading drift: rewrote README.md to reflect the real Sprint 1 state (4 working Lambda handlers, not stubs), corrected handler descriptions in CLAUDE.md, and removed references to files that never landed (dev-client-setup.md, practice-feedback.tsx).
- Enabled Expo DevClient alongside Expo Go to unblock Sprint 3 work (S3 audio uploads) that would otherwise break Expo Go's native-module sandbox.
- Implemented in-session audio recording (ENGAGE-06): facilitator can record the story locally on device with a visible elapsed timer, giving the Engage flow its core capture step.
- Implemented post-recording transcription (ENGAGE-07) so each session produces a text version of the story, plus an accuracy-confirmation step letting the facilitator verify the transcript matches the audio.
- Led the creation of the Timeslips App v1.0 release notes and installation guide, and sent off the final client handoff.

Contact Information: esanint3@gatech.edu

Member 3

Name: Joel Sari

Role: Full Stack Developer

Technical Contributions:

- Led UI/UX polish across the app based on client feedback (layout, spacing, hierarchy, consistency) to prep for demo quality.
- Standardized theming/design system using `constants/theme.ts + useTheme() / tw()` so screens/components match across themes.
- Owned key Home screen work (`tabs/index.tsx`): Prompt of the Day + detail modal flow, Recent Sessions behavior, and Session Stats (migrated from static to backend-driven averages with accurate star rendering).
- Implemented and refined Analytics UI: fixed plot/label alignment and improved readability (axis sizing, spacing, chart height).
- Built the Average Stats carousel with auto-scroll and navigation into the Analytics view.
- Reworked Prompt UI/metadata display: PromptCard + PromptDetailModal to properly separate Category vs Tags, keep styling consistent, and improve modal actions (favorite + close).
- Added prompt library image support (rendering + backend-compatible image URL/fallback behavior).

- Integrated Appwrite auth into the mobile flow (JWT + authenticated API request pattern).
- Contributed to overall navigation architecture (tab bar UX + routing across Home/Engage/Prep/History/Profile).
- Produced/iterated Figma designs for key screens/components and translated them into reusable implementation.

Contact Information: jsari6@gatech.edu

Member 4

Name: Joshua Dietzenbach

Role: Front End Developer

Technical Contributions:

- Primarily a UI/UX front-end developer and designer
- Co-produced Figma designs for implementation.
- Created the prompt, session rating, and session cards.
- Designed and created the session creation screen, including all sections and organization options.
- Created the recording, transcription, reflection, and confirmation screen, and all elements within it.
- Designed and created the prep, browse, and shortlist page, including creating the ability to search by name.
- Designed and created the history screen, including the ability to search for sessions and filter by date.
- Designed and created the session details and the delete session screen.
- Created many various UI components that were reused throughout the app, including search bars, the star rating system, and various buttons.

Contact Information: jdietzenbach3@gatech.edu

Member 5

Name: Viktoriya Petrova

Role: Backend Developer / Project Planning Contributor

Technical Contributions:

- Created and maintained the team's Iteration Plan across Sprint 1, Sprint 2, and Sprint 3, updating sprint goals, user stories, task breakdowns, acceptance criteria, and demo-related progress based on Jira and sprint feedback.

- Contributed to the UI blueprint and planning for both Engage Mode and the Admin Dashboard, helping define how facilitator workflows, admin-only actions, and backend-connected screens should be organized.

- Designed and implemented the Sprint 1 Admin Dashboard, including the admin-facing user-management flow for viewing users, changing roles, and enabling or disabling accounts.

- Created and configured AWS/DynamoDB table support for admin dashboard entries and user-management data, including role information and account enabled/disabled status.
- Implemented backend support for admin user-management actions, including role changes between Admin and Facilitator and account enable/disable updates.
- Integrated Amazon SES email notifications for account-related events, including when validated email accounts update account details and when an admin changes a user's role or account status.
- Created backend support for Engage Mode using AWS resources, including DynamoDB tables and backend function logic for storing and retrieving Engage-related data.
- Implemented the Sprint 3 ADMIN-01 session deletion request feature, allowing facilitators to request deletion of a session and allowing admins to view those requests in the Admin Dashboard.
- Added backend persistence for session deletion requests and connected the request flow to the Admin Dashboard so admins could review submitted deletion requests.
- Added Amazon SES email notifications for session deletion requests so administrators are alerted when a new request is submitted.

Contact Information: vpetrova3@gatech.edu

Section 2: API Specifications

Endpoint 1: Appwrite Service Interface

Method: Appwrite SDK (REST / WebSocket)

Path: https://[Host-Endpoint]/v1

Description: This interface serves as the centralized communication layer between the React Native frontend and the backend services. It abstracts standard RESTful operations to handle Authentication (OAuth/Email), Database Management (NoSQL CRUD), and File Storage. All requests are routed through the Appwrite Client, which manages persistent sessions and real-time state synchronization via WebSockets.

Request Parameters: * Header - Project ID: Unique identifier for the Appwrite project environment.

Header - Secret/JWT: Authentication token or API key for secure request validation.

Body: Payload objects specific to the service being accessed (e.g., user credentials for Auth or document attributes for Database operations).

Response Format: ``json { "\$id": "string", "\$createdAt": "timestamp", "\$updatedAt": "timestamp", "\$permissions": ["string"], "data": { "custom_fields": "application_specific_values" } }